Lab 2: Calculus, Simulation, and Robust Inference

SOCIOL 723 — Social Statistics II — Thursday, September 3

Wenhao Jiang

Table of contents

1 Goals

1. Review the calculus we actually use: derivatives, gradients, the chain rule, and numerical optimization. This is the bridge to maximum likelihood next week.
2. See a sampling distribution by *building* one, through simulation.
3. Watch classical standard errors fail under heteroskedasticity, and watch robust standard errors fix them.
4. Compute HC0–HC3 and cluster-robust standard errors in `R`, by hand and with packages.
5. Meet the few-clusters problem, and repair it.
6. Bootstrap, from first principles and with `boot`.

```
library(dplyr)
library(ggplot2)
library(sandwich)      # vcovHC(), vcovCL()
library(lmtest)        # coeftest(), waldtest()
library(clubSandwich)  # CR2 + Satterthwaite
library(boot)          # bootstrap
library(fixest)        # fast fixed effects with clustered SEs
```

```
gss <- readRDS("../Data/gss_earnings.rds")
c(n = nrow(gss), psu = n_distinct(gss$psu),
  stratum = n_distinct(gss$stratum), region = n_distinct(gss$region))
#>       n     psu stratum  region
#>    3509     839     454       9
```

2 Calculus, as we use it

Almost all of estimation is: *write down a criterion function, differentiate it, set the derivative to zero, and solve.* Last week the criterion was $S(b) = \sum_i (Y_i - X_i'b)^2$; next week it is a log-likelihood. The mechanics are identical.

2.1 Derivatives, numerically

```
f  <- function(x) x^3 - 4 * x + 1        # a scalar function
fp <- function(x) 3 * x^2 - 4            # its analytic derivative

## central difference approximation
num_deriv <- function(f, x, h = 1e-5) (f(x + h) - f(x - h)) / (2 * h)

x0 <- 1.3
c(analytic = fp(x0), numeric = num_deriv(f, x0))
#> analytic  numeric
#>     1.07     1.07
```

The central difference $(f(x+h) - f(x-h))/2h$ has error $O(h^2)$; the forward difference $(f(x+h) - f(x))/h$ has error only $O(h)$. Use central differences when you code your own.

2.2 Gradients

For a function of several arguments, the gradient stacks the partial derivatives. Here is the OLS criterion in the bivariate case:

$$S(b_0, b_1) = \sum_i (Y_i - b_0 - b_1 X_i)^2, \qquad \nabla S = \begin{pmatrix} -2\sum_i (Y_i - b_0 - b_1 X_i) \\ -2\sum_i X_i (Y_i - b_0 - b_1 X_i) \end{pmatrix}.$$

```
y <- gss$lnearn; x <- gss$educ

S <- function(b) sum((y - b[1] - b[2] * x)^2)

grad_S <- function(b) {
  r <- y - b[1] - b[2] * x
  c(-2 * sum(r), -2 * sum(x * r))
}

## numeric gradient, for checking
num_grad <- function(f, b, h = 1e-6) {
  sapply(seq_along(b), function(j) {
    e <- rep(0, length(b)); e[j] <- h
    (f(b + e) - f(b - e)) / (2 * h)
  })
}

b_test <- c(8, 0.1)
rbind(analytic = grad_S(b_test), numeric = num_grad(S, b_test))
#>           [,1]   [,2]
#> analytic -3523 -52062
#> numeric  -3523 -52062
```

> **Tip**
>
> Always check an analytic gradient against a numeric one before trusting it. In Week 3 you will hand-code log-likelihood gradients, and this is the only reliable way to catch a sign error.

2.3 Optimization

`optim()` searches for the minimum of a function. Give it the gradient and it converges faster and more accurately.

```r
fit_optim <- optim(par = c(0, 0), fn = S, gr = grad_S,
                   method = "BFGS", control = list(reltol = 1e-14))

rbind(optim = fit_optim$par,
      lm    = unname(coef(lm(lnearn ~ educ, data = gss))))
#>         [,1]   [,2]
#> optim 8.231 0.1187
#> lm    8.231 0.1187
```

The OLS problem is *convex*, so `optim()` finds the same answer as the closed form from any starting point. Most likelihoods are not so friendly — which is why next week we care about starting values, convergence codes, and the Hessian.

```r
## the Hessian of S is 2 X'X; its inverse scaled by s^2 is Var(bhat)
H <- optim(par = fit_optim$par, fn = S, gr = grad_S, method = "BFGS",
           hessian = TRUE)$hessian
X <- model.matrix(~ educ, data = gss)
all.equal(H, 2 * crossprod(X), check.attributes = FALSE, tolerance = 1e-4)
#> [1] TRUE
```

3 Building a sampling distribution

The fastest way to understand a standard error is to construct one.

```r
set.seed(723)

## a population we control completely
sim_once <- function(n = 200, beta1 = 0.5, hetero = FALSE) {
  x <- rnorm(n, mean = 0, sd = 1)
  sd_i <- if (hetero) 0.5 * exp(0.8 * x) else 1     # error sd
  e <- rnorm(n, sd = sd_i)
  y <- 1 + beta1 * x + e
  m <- lm(y ~ x)
  c(b = coef(m)[2],
    se_classical = sqrt(diag(vcov(m)))[2],
    se_hc3 = sqrt(diag(vcovHC(m, type = "HC3")))[2])
}

reps_homo <- t(replicate(4000, sim_once(hetero = FALSE)))
head(round(reps_homo, 4), 3)
#>         b.x se_classical.x se_hc3.x
#> [1,] 0.4439         0.0671   0.0649
#> [2,] 0.5575         0.0770   0.0782
```

```
#> [3,] 0.5671          0.0789    0.0752
```

```
tibble::tibble(b = reps_homo[, 1]) |>
  ggplot(aes(b)) +
  geom_histogram(bins = 50, fill = "navy", alpha = .65) +
  geom_vline(xintercept = 0.5, colour = "firebrick", linewidth = .7) +
  labs(x = expression(hat(beta)[1]), y = "count") +
  theme_minimal(base_size = 10)
```

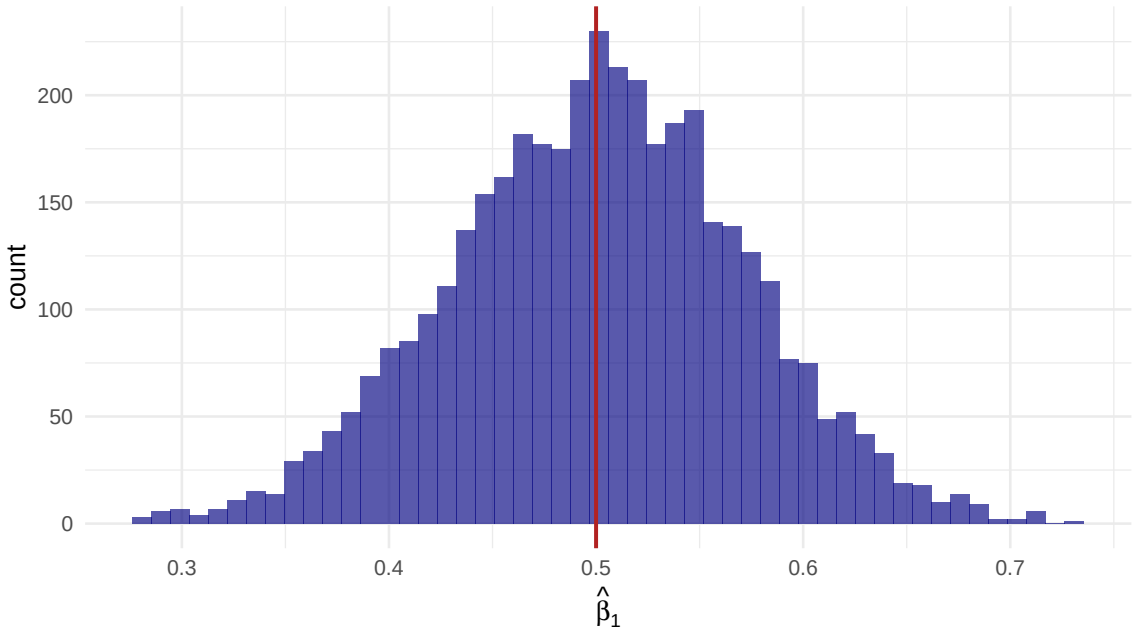

Figure 1: Sampling distribution of $\hat{\beta}_1$ over 4,000 samples of size 200. The true value is 0.5.

The *true* standard error is the standard deviation of this histogram. Compare it to what the two formulas report on average:

```
c(truth = sd(reps_homo[, "b.x"]),
  mean_classical = mean(reps_homo[, "se_classical.x"]),
  mean_hc3 = mean(reps_homo[, "se_hc3.x"]))
#>          truth mean_classical       mean_hc3
#>        0.07095        0.07102        0.07165
```

Under homoskedasticity both are right; the robust one is very slightly noisier. That is the small price we pay for insurance. Now turn heteroskedasticity on:

```
reps_het <- t(replicate(4000, sim_once(hetero = TRUE)))
c(truth = sd(reps_het[, "b.x"]),
  mean_classical = mean(reps_het[, "se_classical.x"]),
  mean_hc3 = mean(reps_het[, "se_hc3.x"]))
#>          truth mean_classical       mean_hc3
```

```
#>        0.12349        0.06559        0.11578
```

The classical formula is badly wrong; HC3 tracks the truth. What matters in practice is the **coverage** of the resulting confidence intervals:

```
covers <- function(b, se, truth = 0.5, crit = 1.96) {
  mean(abs(b - truth) / se < crit)
}
c(classical = covers(reps_het[, "b.x"], reps_het[, "se_classical.x"]),
  hc3       = covers(reps_het[, "b.x"], reps_het[, "se_hc3.x"]))
#> classical       hc3
#>    0.7167    0.9480
```

A nominal 95% interval that covers 80% of the time is not a 95% interval.

> **!** Important
>
> Note what did **not** happen: $\hat{\beta}_1$ stayed centred on 0.5 in both cases. Heteroskedasticity does not bias the estimate. It corrupts the *stated uncertainty*.

4 Robust standard errors on real data

4.1 By hand

The scalar formula from lecture, for the bivariate case:

$$\widehat{\mathrm{Var}}_{\mathrm{HC0}}(\hat{\beta}_1) = \frac{\sum_i d_i^2 \hat{e}_i^2}{\left(\sum_i d_i^2\right)^2}, \qquad d_i = X_i - \bar{X}.$$

```
m_biv <- lm(lnearn ~ educ, data = gss)
d  <- gss$educ - mean(gss$educ)
eh <- residuals(m_biv)

se_hc0_manual <- sqrt(sum(d^2 * eh^2) / (sum(d^2))^2)
se_hc0_pkg    <- sqrt(diag(vcovHC(m_biv, type = "HC0")))["educ"]
se_classical  <- sqrt(diag(vcov(m_biv)))["educ"]

round(c(classical = se_classical, hc0_manual = se_hc0_manual,
        hc0_sandwich = se_hc0_pkg), 6)
#>    classical.educ        hc0_manual hc0_sandwich.educ
#>          0.005213          0.005416          0.005416
```

The general matrix version, also by hand:

```r
m <- lm(lnearn ~ educ + exper + I(exper^2) + female + black, data = gss)
X <- model.matrix(m); e <- residuals(m)
bread <- solve(crossprod(X))
meat  <- crossprod(X * e)          # = sum_i e_i^2 x_i x_i'
V_hc0 <- bread %*% meat %*% bread
round(cbind(manual = sqrt(diag(V_hc0)),
            sandwich = sqrt(diag(vcovHC(m, type = "HC0")))), 6)
#>               manual sandwich
#> (Intercept) 0.105397 0.105397
#> educ        0.005475 0.005475
#> exper       0.005696 0.005696
#> I(exper^2)  0.000118 0.000118
#> female      0.028126 0.028126
#> black       0.038527 0.038527
```

4.2 HC0 through HC3

```r
ses <- sapply(c("const", "HC0", "HC1", "HC2", "HC3"),
              function(ty) sqrt(diag(vcovHC(m, type = ty))))
round(ses, 5)
#>               const     HC0     HC1     HC2     HC3
#> (Intercept) 0.09903 0.10540 0.10549 0.10556 0.10572
#> educ        0.00510 0.00547 0.00548 0.00548 0.00549
#> exper       0.00546 0.00570 0.00570 0.00571 0.00572
#> I(exper^2)  0.00011 0.00012 0.00012 0.00012 0.00012
#> female      0.02793 0.02813 0.02815 0.02815 0.02818
#> black       0.03856 0.03853 0.03856 0.03858 0.03864
```

```r
coeftest(m, vcov = vcovHC(m, type = "HC3"))
#>
#> t test of coefficients:
#>
#>              Estimate Std. Error t value             Pr(>|t|)
#> (Intercept)  7.390108   0.105715   69.91 < 0.0000000000000002 ***
#> educ         0.140532   0.005490   25.60 < 0.0000000000000002 ***
#> exper        0.052875   0.005716    9.25 < 0.0000000000000002 ***
#> I(exper^2)  -0.000739   0.000118   -6.26        0.00000000044 ***
#> female      -0.375785   0.028177  -13.34 < 0.0000000000000002 ***
#> black       -0.191473   0.038636   -4.96        0.00000075431 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

With $n = 3509$ the four robust variants are nearly identical. Rerun this on a subsample of 100

and the gaps open up:

```
set.seed(1)
small <- gss[sample(nrow(gss), 100), ]
m_small <- lm(lnearn ~ educ + exper + I(exper^2) + female + black, data = small)
round(sapply(c("const", "HC0", "HC1", "HC2", "HC3"),
             function(ty) sqrt(diag(vcovHC(m_small, type = ty)))["educ"]), 5)
#> const.educ   HC0.educ   HC1.educ   HC2.educ   HC3.educ
#>    0.02291    0.02291    0.02363    0.02434    0.02600
```

4.3 Robust joint tests

A trap worth internalizing: `anova()` is **not** robust.

```
m_long <- lm(lnearn ~ educ + exper + I(exper^2) + female + black +
               wordsum + pareduc, data = gss)
m_short <- lm(lnearn ~ educ + exper + I(exper^2) + female + black, data = gss)

anova(m_short, m_long)                                          # NOT robust
#> Analysis of Variance Table
#>
#> Model 1: lnearn ~ educ + exper + I(exper^2) + female + black
#> Model 2: lnearn ~ educ + exper + I(exper^2) + female + black + wordsum +
#>     pareduc
#>   Res.Df  RSS Df Sum of Sq    F           Pr(>F)
#> 1   3503 2350
#> 2   3501 2313  2      37.5 28.4 0.00000000000061 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
waldtest(m_short, m_long, vcov = vcovHC(m_long, type = "HC3")) # robust
#> Wald test
#>
#> Model 1: lnearn ~ educ + exper + I(exper^2) + female + black
#> Model 2: lnearn ~ educ + exper + I(exper^2) + female + black + wordsum +
#>     pareduc
#>   Res.Df Df    F           Pr(>F)
#> 1   3503
#> 2   3501  2 26.9 0.0000000000026 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Same null hypothesis, two different tests. Report the second.

5 Clustering

The GSS is a multi-stage area probability sample: it draws primary sampling units (`psu`) within strata (`stratum`), then households within PSUs. Respondents in the same PSU live in the same neighbourhood, so their errors are correlated.

```
ses_cl <- c(
  classical   = sqrt(diag(vcov(m)))["educ"],
  hc3         = sqrt(diag(vcovHC(m, type = "HC3")))["educ"],
  cluster_psu = sqrt(diag(vcovCL(m, cluster = ~ psu)))["educ"],
  cluster_str = sqrt(diag(vcovCL(m, cluster = ~ stratum)))["educ"],
  cluster_yr  = sqrt(diag(vcovCL(m, cluster = ~ year)))["educ"],
  cluster_reg = sqrt(diag(vcovCL(m, cluster = ~ region)))["educ"]
)
round(ses_cl, 5)
#>   classical.educ         hc3.educ cluster_psu.educ cluster_str.educ
#>          0.00510          0.00549          0.00561          0.00550
#>  cluster_yr.educ cluster_reg.educ
#>          0.00497          0.00458
```

Two things to notice.

1. Clustering on `psu` (839 clusters) raises the standard error modestly. GSS PSUs contain few respondents each, so the Moulton inflation $1 + (\bar{n} - 1)\rho_x\rho_e$ is small.
2. Clustering on `region` (9 clusters) *lowers* it. **This is not good news.** With 9 clusters the cluster-robust estimator is severely biased downward. A smaller standard error from coarser clustering is a warning sign, not a result.

5.1 The intra-class correlation

```
## one-way ANOVA ICC with the unequal-cluster-size correction n0
icc <- function(resid, cluster) {
  df <- data.frame(r = resid, g = cluster)
  ms <- df |> group_by(g) |> summarise(n = n(), m = mean(r), .groups = "drop")
  N <- nrow(df); G <- nrow(ms)
  between <- sum(ms$n * (ms$m - mean(df$r))^2) / (G - 1)
  within  <- sum((df$r - ms$m[match(df$g, ms$g)])^2) / (N - G)
  n0 <- (N - sum(ms$n^2) / N) / (G - 1)      # NOT mean cluster size
  (between - within) / (between + (n0 - 1) * within)
}
c(psu = icc(residuals(m), gss$psu),
  region = icc(residuals(m), gss$region))
#>      psu   region
#> 0.061330 0.006255
```

5.2 Simulating the few-clusters problem

```
set.seed(723)

sim_cluster <- function(G = 10, n_g = 50, rho = 0.3) {
  g   <- rep(1:G, each = n_g)
  nu  <- rnorm(G, sd = sqrt(rho))[g]        # cluster shock
  eta <- rnorm(G * n_g, sd = sqrt(1 - rho)) # idiosyncratic
  x   <- rep(rnorm(G), each = n_g)          # treatment varies at cluster level
  y   <- 0 * x + nu + eta                   # TRUE EFFECT IS ZERO
  d   <- data.frame(y, x, g)
  mm  <- lm(y ~ x, data = d)
  b   <- coef(mm)[2]
  c(
    naive   = abs(b / sqrt(diag(vcov(mm)))[2]),
    cr1     = abs(b / sqrt(diag(vcovCL(mm, cluster = ~ g)))[2]),
    cr2_t   = abs(coef_test(mm, vcov = "CR2", cluster = d$g,
                            test = "Satterthwaite")$tstat[2]),
    cr2_p   = coef_test(mm, vcov = "CR2", cluster = d$g,
                        test = "Satterthwaite")$p_Satt[2]
  )
}

out <- t(replicate(600, sim_cluster(G = 10)))
c(
  "reject rate, naive SE"     = mean(out[, "naive.x"] > 1.96),
  "reject rate, CR1"          = mean(out[, "cr1.x"]  > 1.96),
  "reject rate, CR2 + Satt."  = mean(out[, "cr2_p"]  < 0.05)
)
#>    reject rate, naive SE         reject rate, CR1 reject rate, CR2 + Satt.
#>                   0.6583                   0.1717                   0.0400
```

All three should reject 5% of the time — the true effect is zero. The naive standard error rejects far too often; CR1 with 10 clusters still over-rejects; CR2 with Satterthwaite degrees of freedom is close to nominal.

```
coef_test(m, vcov = "CR2", cluster = gss$region, test = "Satterthwaite")[1:3, ]
#> Alternative hypothesis: two-sided
#>        Coef. Estimate      SE Null value t-stat d.f. (Satt) p-val (Satt) Sig.
#>  (Intercept)   7.3901 0.07542          0   98.0        6.67       <0.001  ***
#>         educ   0.1405 0.00455          0   30.9        6.66       <0.001  ***
#>        exper   0.0529 0.00350          0   15.1        6.75       <0.001  ***
```

Look at the `df_Satt` column: with 9 regions, the effective degrees of freedom are in single digits,

so the critical value is far above 1.96.

6 The bootstrap

6.1 By hand

```
set.seed(723)
B <- 2000
boot_b <- replicate(B, {
  idx <- sample(nrow(gss), replace = TRUE)     # resample PAIRS
  coef(lm(lnearn ~ educ + exper + I(exper^2) + female + black,
          data = gss[idx, ]))["educ"]
})

c(estimate = coef(m)["educ"],
  se_boot  = sd(boot_b),
  se_hc3   = sqrt(diag(vcovHC(m, type = "HC3")))["educ"])
#> estimate.educ       se_boot   se_hc3.educ
#>      0.140532      0.005508      0.005490

quantile(boot_b, c(.025, .975))       # percentile interval
#>   2.5%  97.5%
#> 0.1300 0.1515
```

The pairs bootstrap resamples (Y_i, X_i) jointly, so it makes no assumption about σ_i^2. Asymptotically it targets the same sandwich variance that the HC estimators do, which is why it lands in the same neighbourhood — the small gaps among HC0/HC3/bootstrap here are finite-sample, not conceptual.

```
tibble::tibble(b = boot_b) |>
  ggplot(aes(b)) +
  geom_histogram(bins = 50, fill = "navy", alpha = .65) +
  geom_vline(xintercept = coef(m)["educ"], colour = "firebrick") +
  labs(x = "bootstrap replicate of the coefficient on educ", y = "count") +
  theme_minimal(base_size = 10)
```

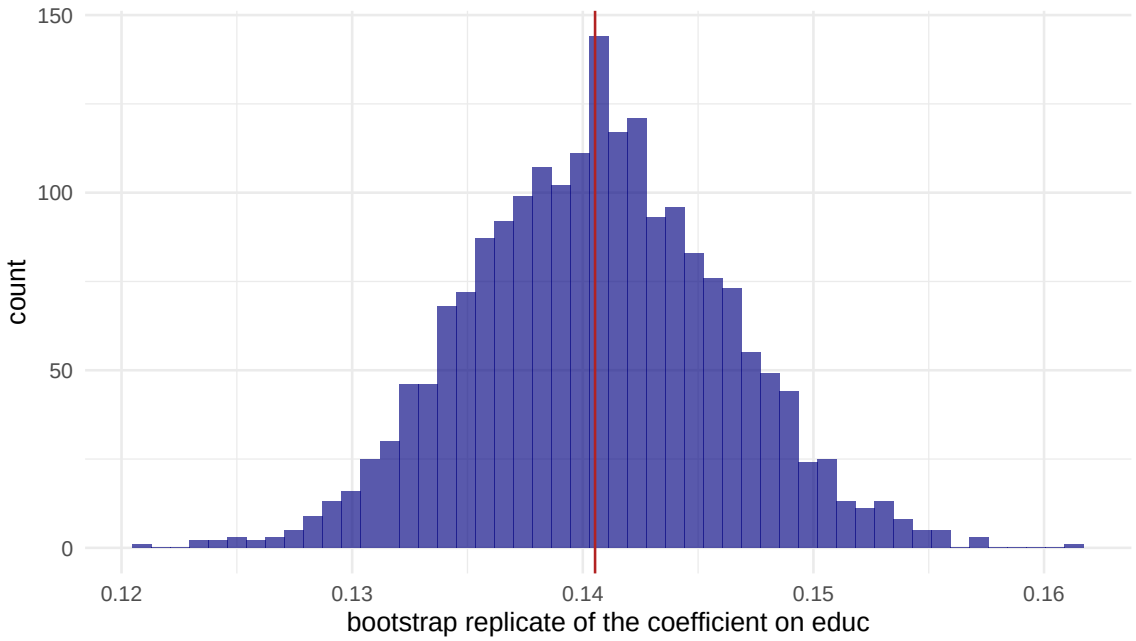

Figure 2: Bootstrap distribution of the coefficient on years of schooling, $B = 2{,}000$.

6.2 With the `boot` package, including BCa

```
b_fun <- function(data, idx) {
  unname(coef(lm(lnearn ~ educ + exper + I(exper^2) + female + black,
                 data = data[idx, ]))["educ"])
}
set.seed(723)
bo <- boot(gss, b_fun, R = 2000)
boot.ci(bo, type = c("norm", "perc", "bca"),
        L = empinf(bo, type = "jack"))
#> BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
#> Based on 2000 bootstrap replicates
#>
#> CALL :
#> boot.ci(boot.out = bo, type = c("norm", "perc", "bca"), L = empinf(bo,
#>     type = "jack"))
#>
#> Intervals :
#> Level      Normal             Percentile            BCa
#> 95%   ( 0.1296,  0.1513 )   ( 0.1296,  0.1512 )   ( 0.1294,  0.1510 )
#> Calculations and Intervals on Original Scale
```

> **Note**
>
> The BCa interval needs *empirical influence values* — roughly, how much each observation pulls the estimate — to compute its acceleration constant. The default way `boot` estimates them (a regression approximation) fails here, so we supply the jackknife version explicitly with `empinf(bo, type = "jack")`. If you ever see `estimated adjustment 'a' is NA`, this is the fix.

6.3 The cluster bootstrap

Resample **clusters**, not individuals:

```
set.seed(723)
psus <- unique(gss$psu)
boot_cl <- replicate(1000, {
  draw <- sample(psus, length(psus), replace = TRUE)
  dat  <- do.call(rbind, lapply(draw, function(p) gss[gss$psu == p, ]))
  coef(lm(lnearn ~ educ + exper + I(exper^2) + female + black,
          data = dat))["educ"]
})
c(cluster_boot = sd(boot_cl),
  analytic_CR1 = sqrt(diag(vcovCL(m, cluster = ~ psu)))["educ"])
#>      cluster_boot analytic_CR1.educ
#>          0.005585          0.005611
```

> **Warning**
>
> If you had resampled *individuals* here, the bootstrap would have reproduced the too-small non-clustered standard error just as faithfully as the analytic formula does. The bootstrap does not know your design — you have to tell it.

7 A convenient shortcut: `fixest`

For everyday work, `fixest::feols()` computes clustered standard errors on the fly and is very fast.

```
fe <- feols(lnearn ~ educ + exper + I(exper^2) + female + black | year,
            data = gss, cluster = ~ psu)
summary(fe)
#> OLS estimation, Dep. Var.: lnearn
#> Observations: 3,509
#> Fixed-effects: year: 6
#> Standard-errors: Clustered (psu)
#>             Estimate Std. Error t value        Pr(>|t|)
```

```
#> educ        0.138413   0.005593  24.745       < 2.2e-16 ***
#> exper       0.053594   0.006032   8.885       < 2.2e-16 ***
#> I(exper^2) -0.000752   0.000123  -6.111 0.0000000015165 ***
#> female     -0.373403   0.026991 -13.834       < 2.2e-16 ***
#> black      -0.195537   0.040129  -4.873 0.0000013160266 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#> RMSE: 0.813625     Adj. R2: 0.239282
#>                  Within R2: 0.231064
```

The `| year` term absorbs year fixed effects (FWL under the hood, as in Lab 1), and `cluster = ~ psu` supplies the cluster-robust variance.

8 Exercises

1. **Coverage of the classical interval.** Repeat the coverage simulation with error standard deviation $0.5\exp(1.5x_i)$ instead of $0.5\exp(0.8x_i)$. How bad does classical coverage get? Does HC3 hold up?
2. **HC3 versus HC0 in small samples.** For $n \in \{30, 60, 120, 500\}$, simulate 2,000 heteroskedastic datasets and record the coverage of intervals built from HC0 and from HC3. At what n does the difference stop mattering?
3. **The Moulton factor.** In `sim_cluster()`, vary $\rho \in \{0, 0.05, 0.2, 0.5\}$ with $G = 50$ and $\bar{n} = 50$. Compare the ratio of the clustered to the naive standard error against the Moulton prediction $\sqrt{1 + (\bar{n} - 1)\rho_x\rho_e}$ with $\rho_x = 1$.
4. **Choosing the cluster level.** Suppose you wanted to estimate the effect of a state-level minimum-wage law on individual earnings, using individual GSS data. At what level would you cluster, and why? What if the law varied by state *and* year?
5. **Delta method versus bootstrap.** Estimate the experience profile's turning point $-\hat{\beta}_{\text{exper}}/(2\hat{\beta}_{\text{exper}^2})$. Get its standard error two ways: `car::deltaMethod()` and a pairs bootstrap. Do they agree? Which interval would you report?
6. **A robust F-test by hand.** Using $\boldsymbol{R}$, $\widehat{\text{Var}}(\hat{\boldsymbol{\beta}})$ and the Wald formula from lecture, compute the joint test that `wordsum` and `pareduc` are both zero, with an HC3 variance. Confirm it matches `waldtest()`.

9 Session information

```
sessionInfo()
#> R version 4.4.1 (2024-06-14)
#> Platform: aarch64-apple-darwin20
#> Running under: macOS 26.5.2
```

```
#>
#> Matrix products: default
#> BLAS:   /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRblas.0.dylib
#> LAPACK: /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRlapack.dylib;
#>
#> locale:
#> [1] C.UTF-8/C.UTF-8/C.UTF-8/C/C.UTF-8/C.UTF-8
#>
#> time zone: Asia/Shanghai
#> tzcode source: internal
#>
#> attached base packages:
#> [1] stats     graphics  grDevices utils     datasets  methods   base
#>
#> other attached packages:
#> [1] fixest_0.12.1      boot_1.3-30        clubSandwich_0.6.0 lmtest_0.9-40
#> [5] zoo_1.8-12         sandwich_3.1-1     ggplot2_4.0.0      dplyr_1.1.4
#>
#> loaded via a namespace (and not attached):
#>  [1] utf8_1.2.4          generics_0.1.3      dreamerr_1.5.0
#>  [4] lattice_0.22-7      digest_0.6.37       magrittr_2.0.3
#>  [7] evaluate_1.0.0      grid_4.4.1          estimability_1.5.1
#> [10] RColorBrewer_1.1-3  mvtnorm_1.3-1       fastmap_1.2.0
#> [13] jsonlite_1.8.9      Matrix_1.7-0        Formula_1.2-5
#> [16] tinytex_0.53        survival_3.6-4      multcomp_1.4-29
#> [19] fansi_1.0.6         scales_1.4.0        TH.data_1.1-4
#> [22] stringmagic_1.2.0   codetools_0.2-20    numDeriv_2016.8-1.1
#> [25] cli_3.6.5           rlang_1.3.0         splines_4.4.1
#> [28] withr_3.0.1         yaml_2.3.10         tools_4.4.1
#> [31] coda_0.19-4.1       vctrs_0.6.5         R6_2.5.1
#> [34] lifecycle_1.0.4     emmeans_1.11.1      MASS_7.3-60.2
#> [37] pkgconfig_2.0.3     pillar_1.9.0        gtable_0.3.6
#> [40] glue_1.7.0          Rcpp_1.1.1          xfun_0.47
#> [43] tibble_3.2.1        tidyselect_1.2.1    knitr_1.48
#> [46] xtable_1.8-4        farver_2.1.2        htmltools_0.5.8.1
#> [49] nlme_3.1-164        labeling_0.4.3      rmarkdown_2.28
#> [52] compiler_4.4.1      S7_0.2.0
```